

FARM MANAGEMENT SYSTEM

Project Documentation

Date: 1 October 2026

Team ID: 62ab296c9dbd693aadc

A web-based system for managing farm activities, crops, resources, workers, expenses and harvest information.

1. Project Description

Farm Management System is a web-based application designed to simplify and organize day-to-day farm operations. The system helps farmers maintain information about crops, fields, farming activities, workers, resources, expenses and harvest records in one place.

The application provides a structured way to add, update, view and manage farm information. It can help reduce manual record keeping and make important farm data easier to access.

Main modules include Crop Management, Field Management, Activity Management, Worker Management, Resource Management, Expense Management and Harvest Management.

Scenarios

- A farmer registers a field and records the crop planted, planting date and expected harvest date.
- A farm manager records activities such as sowing, irrigation, fertilization and pest control for a selected crop.
- A manager records workers assigned to farm activities and tracks the work performed.
- Resources such as seeds, fertilizer and pesticides are recorded with available quantity and usage.
- Expenses and harvest details are entered so the farm can maintain organized production records.

2. Technical Architecture

The Farm Management System can be implemented using a three-tier architecture. The frontend provides the user interface, the backend manages business logic and requests, and the database stores farm records.

- Frontend: HTML, CSS and JavaScript for the user interface.
- Backend: Python with Flask for routing, validation and application logic.
- Database: SQLite/MySQL for storing farmer, field, crop, activity, worker, resource, expense and harvest data.
- Version Control: Git and GitHub for source-code management.
- Development Environment: Python, Flask and a suitable code editor.

Database Note: Since this project uses a database, an ER Diagram should be included to show the relationships between the major entities.

3. Pre-requisites

- Basic Python programming knowledge.
- Flask framework knowledge.
- HTML, CSS and JavaScript fundamentals.
- SQL and database fundamentals.
- Git and GitHub basics.
- Knowledge of CRUD operations and form handling.
- Development environment with Python and required packages installed.

4. Project Workflow

Milestone 1: Requirement Analysis and Architecture

- Identify the main users and farm-management requirements.
- Define modules such as crops, fields, workers, resources, expenses and harvest.
- Design the application architecture and database structure.
- Set up the development environment and project folder structure.

Milestone 2: Database and Core Functionalities

- Create database tables for the main farm entities.
- Implement create, read, update and delete operations.
- Develop crop and field management functionality.
- Develop activity, worker and resource management functionality.

Milestone 3: Backend Development

- Create Flask routes for each module.
- Validate form data and process user requests.
- Connect the Flask application to the database.
- Implement error handling and basic application security.

Milestone 4: Frontend Development

- Design a simple and responsive dashboard.
- Create forms for entering and editing farm records.
- Display crop, activity, resource, expense and harvest information clearly.
- Add navigation and user-friendly messages for successful or failed operations.

Milestone 5: Testing and Deployment

- Test all CRUD operations and database interactions.
- Verify form validation and error handling.
- Test the application on different screen sizes.
- Run the application locally and prepare it for deployment.

Milestone 6: Conclusion

- Review the completed modules and verify that the main farm-management requirements are satisfied.
- Document possible future improvements and scalability options.

5. Milestone 1: Requirement Analysis and Architecture

This milestone focuses on understanding the problems involved in manual farm record keeping and defining the system requirements. The application is planned around centralized management of farm information.

Activity 1.1: Identify System Requirements

Identify information that needs to be stored, including farmer details, fields, crops, activities, workers, resources, expenses and harvest records.

Activity 1.2: Define Modules

Divide the application into manageable modules such as Dashboard, Field Management, Crop Management, Activity Management, Worker Management, Resource Management, Expense Management and Harvest Management.

Activity 1.3: Define Application Architecture

Plan the interaction between the frontend, Flask backend and database. User requests are processed by the backend and the required records are read from or written to the database.

Activity 1.4: Set Up Development Environment

Install Python, Flask, database support and required development tools. Create the project directory and configure version control.

6. Milestone 2: Database and Core Functionalities

The database is the central storage component of the Farm Management System. It maintains organized records and supports CRUD operations.

Suggested Core Tables

Table	Purpose
Farmer	Stores farmer or farm-owner information.
Field	Stores field name, location, area and status.
Crop	Stores crop name, type, season and related field.
Activity	Stores sowing, irrigation, fertilization, spraying and other activities.
Worker	Stores worker details and assignments.
Resource	Stores seeds, fertilizer, pesticides and other farm resources.
Expense	Stores farm-related expenses and amounts.
Harvest	Stores harvested crop, quantity, date and related field.

Core Functions

- Add new farm records.
- View existing records.
- Edit or update records.
- Delete records when required.
- Search and filter records for easier access.

7. Milestone 3: Backend Development

The Flask backend acts as the application layer. It receives requests from the frontend, validates data, performs database operations and returns the required information.

Suggested Routes

- / — Dashboard or home page.
- /fields — View and manage fields.
- /crops — View and manage crops.
- /activities — Manage farm activities.
- /workers — Manage workers.
- /resources — Manage farm resources.
- /expenses — Manage expenses.
- /harvest — Manage harvest records.

Backend responsibilities include request handling, input validation, CRUD operations, database connectivity and error handling.

8. Milestone 4: Frontend Development

The frontend should provide a simple dashboard so users can quickly access important farm information.

- Dashboard showing summary information such as total fields, active crops, recent activities, expenses and harvest records.
- Forms for adding and editing farm data.
- Tables or cards for displaying records.
- Navigation menu for moving between modules.
- Responsive design for desktop and mobile screens.

9. Database / ER Diagram Description

The ER Diagram should represent the relationships among the major entities. A Farmer can manage multiple Fields. A Field can contain multiple Crops and can have multiple Activities. Workers can be assigned to Activities. Resources can be used during Activities. Expenses can be recorded for farm operations. Harvest records can be associated with a Crop and Field.

Recommended relationships: Farmer 1-to-many Field; Field 1-to-many Crop; Crop 1-to-many Activity; Worker 1-to-many Activity; Resource 1-to-many Activity; Field/Crop 1-to-many Harvest; Farm/Crop/Activity 1-to-many Expense.

10. Milestone 5: Testing and Deployment

Testing verifies that each module works correctly and that data is stored and displayed properly.

- Test valid and invalid form inputs.
- Verify that new records are inserted correctly.

- Verify that update and delete operations work correctly.
- Check database relationships and displayed values.
- Test navigation and responsive layout.
- Run the application locally before deployment.

Typical local execution can be performed after installing dependencies and configuring the database. The exact deployment method depends on the hosting environment selected for the project.

11. Exploring the Website Pages

Home / Dashboard Page

The dashboard provides an overview of the farm and quick access to the main modules.

Field Management Page

Users can add, view, update and manage fields, including area and status information.

Crop Management Page

Users can record crops, planting information and crop-related details.

Activity Management Page

Users can record farming activities such as irrigation, sowing, fertilization and pest control.

Resource and Expense Pages

These pages help maintain resource usage and financial records.

Harvest Page

Users can record harvested crop details, quantity and harvest date.

12. Future Enhancements

- User authentication and role-based access.
- Weather information integration.
- Crop disease detection using AI or image analysis.
- Automatic reminders for irrigation, fertilizer and harvest schedules.
- Mobile-friendly progressive web application.
- Reports and charts for expenses, production and crop performance.
- Cloud database and online backup.
- Multi-language support.

13. Conclusion

Farm Management System provides a centralized platform for organizing farm operations and records. By combining a user-friendly frontend, Flask-based backend and database storage, the system can reduce manual record keeping and make farm information easier to manage. The project demonstrates practical use of web development, database management and CRUD operations in an agricultural domain.

The system can be expanded in the future with authentication, analytics, weather services, AI-based crop assistance and mobile support.